VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
GAURAV BHARDWAJ	24BSA10187
Slot	**Date of Submission**
C11	31 MARCH,2026

Git	Version Control	GPL v2	*The tool Linus built when proprietary failed him*

1. Introduction

Open source software refers to software whose source code is made freely available to the public, allowing anyone to view, study, modify, and distribute it. Software that has its source code made publicly available so that anyone can view, examine, alter, and distribute it is referred to as open source software. Open source software encourages openness, cooperation, and shared development, in contrast to proprietary software, where the code is constrained and controlled by a single organization. Since many widely used technologies today are based on open source foundations, this model has been instrumental in shaping modern computing.

Software freedom and open source are closely related concepts. The freedom to comprehend how software functions, modify it to suit particular requirements, and share enhancements with others is just as important as having free access to software.

Developers from all over the world can contribute, fix bugs, enhance performance, and add new features thanks to this collaborative approach, frequently at a speed that is challenging for proprietary systems to match.

Because it promotes innovation, lowers costs, and improves accessibility, open source software is important. Technology is more inclusive since both individuals and organizations can use powerful tools without being restricted by licensing. Because source code is available for ongoing review and auditing by a worldwide community, it also improves security and dependability. Additionally, because developers can study real-world codebases and contribute to worthwhile projects, open source promotes learning and skill development.

In todays world open source software is the foundation of many important systems. These systems include operating systems, web servers, databases and development tools. Knowing about open source software is not a choice for computing professionals. It is a must.

This report looks at Git, an open source system that helps track changes in code. We will explore where Git came from, its licensing, what it means for ethics how it is used in systems and its place in the open source software world.

The report will dive into Git and open source software. How they impact the way we work and develop technology. Git and open source are key, to understanding how software gets made and how it works.

2. Project Report Structure

A1. Origin and Problem Statement

The creation of Git is closely tied to the problems faced during the growth of the Linux operating system.In the 2000s the Linux kernel project had grown a lot.Thousands of developers from parts of the world were contributing code.

They needed a system to manage all the code.However the tools available then were either too slow.Some lacked distributed capabilities.Others were proprietary.The Linux developers initially used a version control system called BitKeeper.It had features.It created a problem.The Linux project was source but BitKeeper was not.This became an issue when there were licensing conflicts.Eventually access to BitKeeper was withdrawn.The developers lost trust in tools.Linus Torvalds decided to create a version control system.

It would be free, fast and fully controlled by the developers.He developed Git quickly.Git focused on performance, data integrity and a distributed architecture.Unlike systems Git let every developer have a complete copy of the repository.They did not need to rely on a server.Git was designed to solve problems.It handled codebases quickly.Developers could work offline.They had control over version history.Git ensured that developers would not depend on systems.This was a moment in open-source history.

Control and freedom became priorities.

Today Git is the version control system.It powers platforms like GitHub and GitLab.Millions of developers use Git worldwide.Its origin teaches a lesson.When developers lose control over their tools they build ones to restore that control.Git is a result of this need for control and freedom.It has become a tool, in software development.

A2. Licensing

GNU General Public License version 2 (GPL v2) governs the distribution of Git; it is one of the more prominent types of Red Hat’s open source licenses or software. The GPL v2 was designed to allow people to have access to free software but also to be able to preserve a user’s rights to that software.

The concept of free software under the GPL is based on four essential rights. Users have the right to run the program for any purpose, to read the source code from which the program is derived, to change the program to suit your needs, and to share copies of both the original and modified programs with other individuals. The four essential rights guarantee that the program is kept open and available, can be modified, and is a source for continuing cooperation.

The GNU GPL license's enforcement of the so-called "copyleft" principle is critical to the GPL. In essence, if a corporation or person modifies Git and distributes the altered version, they are obligated to make the source code available under the same license as the original git source code. Consequently, they can not take git, make changes and release a closed-source

derivative. This guarantees that if one party improves git, the improvement will be available to the broader community.

The GNU GPL differs substantially from permissive licenses like MIT in this way. The MIT license permits anyone to use, alter, and redistribute software including incorporated in a proprietary product without the requirement to share any modifications. The GPL favors preserving user freedoms rather than providing businesses with maximum flexibility. The GPL guarantees that all future versions will be open software.

A3. The Ethics of Open Source

Not only is open source a way to build and distribute software using technical methods, but it is also a way to think about how businesses build and share care for each other's knowledge and/or technology. Open source raises ethical issues related to ownership vs. collaboration and the tradeoff between individual effort vs. group benefit.

The primary question to consider is if every piece of software should be open-sourced. In support of open source, open-sourcing software: provides a platform for transparency, provides an opportunity for others (including yourself) to learn from your work, and allows you to give someone access to use the software without any limitations. Thus, all of these benefits accelerate innovation and decrease reliance on large corporations. Conversely, there are instances where certain types of software cannot or should not be developed using an open-source model. For example, while it may be okay for others to use or modify your software, certain types of software (i.e., containing sensitive or personally identifiable information, containing security hardware or software, or containing proprietary business processes) require some level of access restriction. Therefore, while there are many significant benefits of open source, it should not be used as a solution in all instances.

One ethical issue is whether or not it's right for businesses to make money by selling products that are based off of other people's work (this is often referred to as "standing on the shoulders of giants"). There are larger companies that have created many successful products using open-source software (i.e., open-source software, such as git and Linux) who have been able to generate enormous profits by using these tools without giving much back to their communities.

In other instances, however, there are large companies who contribute back to the open-source community through the development and maintenance of this software. The challenge is that it becomes difficult to determine what is fair with respect to the use of open-source software.

The concept of standing on the shoulders of giants is at the heart of open source. That is to say, developers build on other people's work instead of starting from scratch. Open source facilitates this by allowing for the free exchange of information, which allows development

efforts to accelerate and create new and innovative products. With open-source being an option, developers may find less motivation to create completely new systems because they will rather modify an existing platform than building something new.

To sum up, free and open sources form a delicate equilibrium between freedom to act without restriction and assumption of full responsibility for actions taken without restriction by others. Free/open source provides both developers and end-user opportunities for active engagement in the design and construction process, enabling both developers and users to collaborate on common projects, however, this often raises issues related to equity, justice, and sustainability. Free software development has resulted from thousands of programmers working collaboratively toward common goals resulting in global standardization (i.e., Git) demonstrating how collaborative efforts can yield innovative technologies that ultimately gain broad acceptance as standards.

Part B — Linux Footprint (Unit 2)

Part B — System Installation and Linux Footprint

A study of Git was done using the Mac OS Terminal, which runs on a Unix (Darwin) platform. Although Mac OS is not a Linux distribution, it is very similar to it when working from the command line, using the directory structure, and performing system functions. Therefore, Mac OS is a good environment for testing open-source applications.

Git was already on the system because it was installed with the Apple Command Line Tools. The installed version of Git has been confirmed by using command line commands.

Git Version: 2.50.1 (Apple Git-155).

The location of the Git binary (i.e., the executable file for Git) was verified by using the "which" command:

Binary Location: /usr/bin/git.

This indicates that Git is stored in a standard, system-wide directory and is available for access by all users.

Details about the system were retrieved using the "uname" command:

Kernel: Darwin 25.3.0; Architecture: x86_64.

The name of the current user logged into the system is:

User: apple.

The amount of time that the system has been up and running and an indication of its performance were found by using the "uptime" command:

Uptime: 23 days, 8 hours; Load Average: 4.02, 3.20, 3.59.
Initial system date/time

The system date/time is recorded as:

Mon Mar 30 20:01:02 IST 2026

Also, I looked at some important directories to see how the Unix-based environment is set up:

/etc → where the various configuration files are kept

/usr/bin → contains executables (e.g., Git) that we want to run

/var → keeps track of logs and variable data at runtime

I used the ls -ld command to check permissions and ownership of these directories, thus demonstrating how they relate to system security/access control.

Git is a user-level application. It doesn't have a dedicated system user or run as a service process. Users execute Git when they need it and run it in real-time.

Overall, this analysis illustrates Git's integration with the Unix-based operating system and how to investigate open-source software using command-line tools.

```
(base) apple@apples-MacBook-Pro-2 ~ % git --version
git version 2.50.1 (Apple Git-155)
(base) apple@apples-MacBook-Pro-2 ~ % which git
/usr/bin/git
(base) apple@apples-MacBook-Pro-2 ~ % uname -a
Darwin apples-MacBook-Pro-2.local 25.3.0 Darwin Kernel Version 25.3.0: Wed Jan 28 20:53:28 PST 2026; root:xnu-12377.91.3~2/RELEASE_X86_64 x86_64
(base) apple@apples-MacBook-Pro-2 ~ % uptime
22:20  up 23 days, 10:32, 2 users, load averages: 3.05 2.92 2.82
(base) apple@apples-MacBook-Pro-2 ~ % date
Mon Mar 30 22:20:44 IST 2026
(base) apple@apples-MacBook-Pro-2 ~ % ls -ld /etc /usr/bin /var
lrwxr-xr-x@   1 root  wheel     11 25 Feb 09:11 /etc -> private/etc
drwxr-xr-x  923 root  wheel  29536 25 Feb 09:11 /usr/bin
lrwxr-xr-x@   1 root  wheel     11 25 Feb 09:11 /var -> private/var
(base) apple@apples-MacBook-Pro-2 ~ % ls -l /usr/bin/git
-rwxr-xr-x  78 root  wheel  118880 25 Feb 09:11 /usr/bin/git
(base) apple@apples-MacBook-Pro-2 ~ %
```

Part C — The FOSS Ecosystem (Units 3 & 4)

Git should not be viewed as being separated from the larger world of open-source software created around the use of Git. Although Git is a stand-alone distributed version control system, Git relies on core components of a computer and operating system, including the file system, command-line interface and network protocols such as SSH that are used to access remote repositories in order to operate effectively on different operating systems.

While Git as a technology is important to how modern software is created, the biggest impact that Git has left behind is that of the "ecosystem". Various platforms (e.g., GitHub, GitLab) that are built on top of Git have created functionality to improve the process of collaboration among and between developers. Platforms like these (and others) have enabled a change in how groups of developers can work together by providing tools to more easily share code, review others' code, and manage large-scale software development.

Git is a fundamental element of many of today's development processes and is often at the heart of DevOps. It is an extremely versatile tool that can be used for tracking how code has changed over time (version control), facilitating collaboration among developers, and providing deployment pipelines. When developers use Git to collaborate by sharing/reviewing/approving each other's code changes, automated systems can also connect to Git to trigger build, test, and deploy activities via Git hooks. Because of this integration, Git becomes a vital part of the software development life cycle.

In web development, Git is typically just one technology within a larger technology stack; it is not considered part of the traditional LAMP (Linux, Apache, MySQL PHP) stack, but it does play an important role in the development and maintenance of applications that run on traditional lamp

stacks through version control and enabling collaboration. Because Git can facilitate code versioning and collaboration, it supports the overall stability and evolution of web systems.

Git has a vibrant and extensive international community of users and developers who contribute to the continues maintenance of Git by developing new features and fixing bugs. Experienced developers provide the governance of Git through a well-established set of guidelines. The community provides additional support to users through forums, documentation, and other collaborative platforms. The large, diverse and active community surrounding Git will help ensure that Git continues to evolve with new features, improve security and facilitate widespread acceptance.

In conclusion, Git is not only a tool for software developers but is one of the fundamental elements of the open-source development community. Git serves as a hub to connect developers to tools and platforms, thereby allowing collaborative software development to take place anywhere in the world.

Part D — Open Source vs Proprietary (Critical Analysis)

The comparison of Git to proprietary source control systems like Perforce and Bitbucket illustrates the advantages of source control according to the costs for ownership, control, flexibility, and support. Both types of tools are developed for similar problems, but are available at different costs, provide different levels of control and flexibility, and give different levels of support.

Dimension	Open Source	Proprietary Alternative
Cost	Free to use and distribute	Often require paid licenses or subscriptions
Security (who can audit the code?)	Source code is open and can be audited by anyone	Source code is closed;user must trust the company
Support and reliability	Community-driven and support through forums and documentation	Dedicated professional support from the company
Freedom to modify	Full freedom to modify and distribute under GPL lisence	Limited or no freedom to modify the software

Community vs corporate control	Full freedom to modify and distribute under GPL license	Limited or no freedom to modify the software
Your overall verdict	Users have complete control over their repositories	Control is often restricted by vendor policies

The ability to control everything about your project is one of the most significant aspects of Git's use. Developers who use Git are free from being bound to any one organization because it is an open-source program. Developers can create their own repositories, customize the way they work, and use the product in any way they want. Because of these characteristics, Git can be used flexibly and across a broad range of types of projects.

In contrast to open-source tools, proprietary or commercial tools typically provide better support and ease of setting up in corporate environments. Commercial tools often offer additional functions and integrations that can make them simpler to use for nontechnical users; however, using these types of tools may result in less control over your project and can also make it easier for an enterprise vendor to lock you into their service.

Overall, Git is a powerful and flexible option for version control for all developers who value independent work, transparency in their work, and cooperation with their peers. Proprietary tools may serve a purpose in specific corporate contexts; however, Git's more sustainable and adaptable methodologies for long-term development make it an excellent choice. Based on this evaluation, it should be easy to recommend Git for real-world application, and all developers will benefit from and add value to the Git ecosystem.

3. Shell Script Tasks

Script 1 — System Identity Report (Unit 1 & 2)

```
echo "======================================"

echo "Student Name: GAURAV BHARDWAJ"
echo "Registration No: 24BSA10187"

echo "--------------------------------------"

echo "Logged-in User:"
echo "$USER_NAME"

echo "Kernel Version:"
echo "$KERNEL"

echo "System Uptime:"
echo "$UPTIME"

echo "Current Date & Time:"
echo "$DATE"

echo "======================================"
```

```
(base) apple@apples-MacBook-Pro-2 ~ % nano script1.sh
(base) apple@apples-MacBook-Pro-2 ~ % ./script1.sh
uptime: illegal option -- p
usage: uptime
======================================
 SYSTEM IDENTITY REPORT
======================================
Student Name: GAURAV BHARDWAJ
Registration No: 24BSA10187
--------------------------------------
Logged-in User:
apple
Kernel Version:
25.3.0
System Uptime:

Current Date & Time:
Mon Mar 30 23:18:00 IST 2026
======================================
(base) apple@apples-MacBook-Pro-2 ~ %
```

An example script that contains basic system information (including kernel version, current user, system uptime, and date) and uses variables and command substitution to contain output from various system commands (like `uname`, `whoami` and `uptime`). This shows how to automate the reporting of a system using shell scripting and provides structured information.

Script 2 — FOSS Package Inspector (Unit 2)

```
fi

echo "----------------------------------------"

# Description using case
case $PACKAGE in
 git) echo "Description: Git is a distributed version control system used to track changes in code." ;;
 firefox) echo "Description: Firefox is an open-source web browser." ;;
 vlc) echo "Description: VLC is a multimedia player." ;;
 *) echo "Description: Unknown package" ;;
esac

echo "========================================"
```

```
(base) apple@apples-MacBook-Pro-2 ~ % nano script2.sh
(base) apple@apples-MacBook-Pro-2 ~ % ^[[200~chmod +x script2.sh
zsh: bad pattern: ^[[200~chmod
(base) apple@apples-MacBook-Pro-2 ~ % ./script2.sh~
zsh: no such file or directory: ./script2.sh~
(base) apple@apples-MacBook-Pro-2 ~ % chmod +x script2.sh
./script2.sh
========================================
 FOSS PACKAGE INSPECTION REPORT
========================================
Student Name: GAURAV BHARDWAJ
Registration No: 24BSA10187
----------------------------------------
Status: Git is installed
Git Version:
git version 2.50.1 (Apple Git-155)
----------------------------------------
Description: Git is a distributed version control system used to track changes in code.
========================================
(base) apple@apples-MacBook-Pro-2 ~ %
```

The script uses git to determine whether it is present on the computer. If it finds git installed, it will print its current version. It uses a case statement to give a brief description of what the software is, showing both conditional statements and decision making in a shell script.

Script 3 — Disk and Permission Auditor (Unit 2)

```
  UW PICO 5.09                                File: script3.sh                                Mod

  # Get size
  SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)

  echo "Directory: $DIR"
  echo "Permissions & Owner: $PERM"
  echo "Size: $SIZE"
  echo "-------------------------------------"

 else
  echo "Directory $DIR does not exist"
 fi
done

echo "====================================="
```

```
(base) apple@apples-MacBook-Pro-2 ~ % nano script3.sh
(base) apple@apples-MacBook-Pro-2 ~ % ^[[200~chmod +x script3.sh
zsh: bad pattern: ^[[200~chmod
(base) apple@apples-MacBook-Pro-2 ~ % ./script3.sh~
zsh: no such file or directory: ./script3.sh~
(base) apple@apples-MacBook-Pro-2 ~ % chmod +x script3.sh
./script3.sh
=====================================
 DIRECTORY AUDIT REPORT
=====================================
Student Name: GAURAV BHARDWAJ
Registration No: 24BSA10187
-------------------------------------
Directory: /etc
Permissions & Owner: lrwxr-xr-x@ root wheel
Size:   0B
-------------------------------------
Directory: /usr/bin
Permissions & Owner: drwxr-xr-x root wheel
Size:  80M
-------------------------------------
Directory: /var
Permissions & Owner: lrwxr-xr-x@ root wheel
Size:   0B
-------------------------------------
=====================================
(base) apple@apples-MacBook-Pro-2 ~ %
```

Using the `ls`, `du`, and `awk` command line tools, this script loops through all of the relevant system directories to provide information about the permissions, owner, and size of those directories. The use of a loop, conditionals, and command-line tools will help you understand how the system is organized and who has access to certain system files

Script 4 — Log File Analyzer (Units 2 & 5)

Concepts to use: while read loop, if-then, counter variables, command-line arguments ($1).

```
fi

echo "Analyzing file: $LOGFILE"

echo "-------------------------------------"

# Read file line by line
while read LINE
do
 if echo "$LINE" | grep -i "error" > /dev/null
 then
  COUNT=$((COUNT+1))
 fi
done < "$LOGFILE"

echo "Total lines containing 'error': $COUNT"

echo "====================================="
```

```
(base) apple@apples-MacBook-Pro-2 ~ % nano script4.sh
(base) apple@apples-MacBook-Pro-2 ~ % chmod +x script4.sh
./script4.sh /var/log/system.log
=====================================
 LOG FILE ANALYSIS REPORT
=====================================
Student Name: GAURAV BHARDWAJ
Registration No: 24BSA10187
-------------------------------------
Analyzing file: /var/log/system.log
-------------------------------------
Total lines containing 'error': 0
=====================================
(base) apple@apples-MacBook-Pro-2 ~ %
```

This script opens and reads one log entry at a time looking for lines that have the word “error”. It contains a while loop, if statements and command line parameters. It shows how to do basic text processing and automate tasks using shell scripts.

Script 5 — The Open Source Manifesto Generator (Unit 5)

```
echo "----------------------------------------"

read -p "Enter one open-source tool you use: " TOOL
read -p "What does freedom mean to you? " FREEDOM
read -p "What will you build and share? " BUILD

DATE=$(date)
OUTPUT="manifesto.txt"

echo "Generating your manifesto..."

echo "Date: $DATE" > $OUTPUT
echo "I use $TOOL every day." >> $OUTPUT
echo "Freedom means $FREEDOM." >> $OUTPUT
echo "I will build $BUILD and share it openly." >> $OUTPUT

echo "----------------------------------------"
echo "Your manifesto:"
cat $OUTPUT

echo "========================================"
```

```
(base) apple@apples-MacBook-Pro-2 ~ % nano script5.sh
(base) apple@apples-MacBook-Pro-2 ~ % chmod +x script5.sh
(base) apple@apples-MacBook-Pro-2 ~ % ./script5.sh
========================================
 OPEN SOURCE MANIFESTO GENERATOR
========================================
Student Name: GAURAV BHARDWAJ
Registration No: 24BSA10187
----------------------------------------
Enter one open-source tool you use: GIT
What does freedom mean to you? Transparency
What will you build and share? a learning platform
Generating your manifesto...
----------------------------------------
Your manifesto:
Date: Mon Mar 30 23:41:39 IST 2026
I use GIT every day.
Freedom means Transparency.
I will build a learning platform and share it openly.
========================================
(base) apple@apples-MacBook-Pro-2 ~ %
```

The user can generate a personalized country manifesto, also referred to as an open source manifesto, by running this code. This script shows how to take user inputs in an interactive way (using read), manipulate those inputs (using string handling), and write the outputs to files (or redirect the output). This example also illustrates how shell scripts can automate the process of generating content.

CONCLUSION

This audit of Git has been an exercise in tracing the threads that connect philosophy, law, community, and technology. What began as a crisis — a company revoking a license and leaving thousands of developers without their primary tool — produced a piece of software that has become as fundamental to the modern software world as the operating system itself. Git is used by more than 100 million developers. Its distributed model has replaced every previous version control paradigm. GitHub, built on top of it, became the largest software collaboration platform in history. The GPL v2 license that covers Git is not just a legal document — it is an expression of belief: that tools which the community builds together should remain available to the community forever. The four freedoms are not abstract.

Every time a developer forks a repository, every time a Linux distribution ships an update, every time a company builds a product on top of opensource foundations, those freedoms are being exercised. The ethical questions around open source do not resolve cleanly, and they should not. The relationship between corporate use and community contribution is one of the central tensions of the modern technology economy. What this audit has clarified for me is that the answer lies not in a rule but in a disposition: to use open-source software thoughtfully, to understand the labour and philosophy it represents, and to contribute back wherever possible — even if it is only documentation, a bug report, or teaching someone else. The five shell scripts in this project are small demonstrations of the same spirit.

They are not impressive programs. But they are documented, explained, and available for anyone to use, modify, and improve. In that, they are open source — and that, in the end, is the point

4. Reference Resources

1. GNU Project — Free Software Definition
https://www.gnu.org/philosophy/free-sw.html

2. Open Source Initiative — Open Source Definition
https://opensource.org/osd

3. Git Official Documentation
https://git-scm.com/docs

4. GitHub Documentation
https://docs.github.com

5. The Linux Command Line by William Shotts
https://linuxcommand.org

6. GNU Bash Manual
https://www.gnu.org/software/bash/manual

7. SPDX License List
https://spdx.org/licenses

8. Choose a License
https://choosealicense.com

9. Eric S. Raymond — The Cathedral and the Bazaar
http://www.catb.org/~esr/writings/cathedral-bazaa